Three tools and one reliable sender

An abridged version of the three-tools report

A message sent over an unreliable network can be lost, so a sender usually keeps a copy of each message and resends it until the receiver acknowledges it. That remedy is correct, but when acknowledgements are late the sender cannot tell a lost message from a slow one, so it resends messages that have already arrived, and the copies compete with new work for the receiver's capacity.

This report follows such a sender through three tools, each of which asks a different question about it. **Hydrofrac** asks whether the protocol is correct when messages are lost and machines crash. **Cress** asks whether delay makes the protocol do work that its input did not require. **Sedge** asks whether the deployed protocol recovers from a burst of load when the receiver's CPU is capped from outside. The naive protocol passes the first test and fails the other two. We then add a token bucket that limits resends. Only hydrofrac notices that one plausible bucket can lose data forever, and a bucket that refills from both a timer and acknowledgements passes all three tools.

The programs are written in Hydro, a Rust framework in which a distributed program is a *dataflow*: a graph of operators (filters, joins, unions) through which streams of values flow, split across machines that are connected by network channels.

1. Correctness: from `send` to `reliable_send`

Plain `send` over a lossy link

Hydrofrac applies lineage-driven fault injection (LDFI; Alvaro, Rosen and Hellerstein, SIGMOD 2015) to Hydro programs, running them in Hydro's deterministic simulator. A sender passes value 1 to Hydro's ordinary `send` over a channel that may lose messages, and a receiver delivers whatever arrives. LDFI starts from a *successful* run, with no faults, in which the good outcome occurs: the receiver delivers value 1. It then asks whether some combination of failures could falsify that outcome. To answer, it reads the outcome's *lineage*, which is the record of why the outcome happened. Here the lineage is short: the receiver holds 1 because one message carrying 1 crossed the `data` channel. A fault matters only if it cuts that lineage, so LDFI computes the smallest sets of crashes and lost messages that would do so. The smallest is the loss of that one message. Hydrofrac injects the loss and reruns the program, and the receiver never holds 1. That run is a counterexample, found after two fault sets, and Figure 1 shows it as a Lamport diagram, with one timeline per machine and one arrow per message.

The property checked here is validity: *every value handed to a sender that never crashes is delivered at every receiver that never crashes.* If either machine crashes, the property makes no demand. Hydrofrac states a property as an implication from one collection to another:

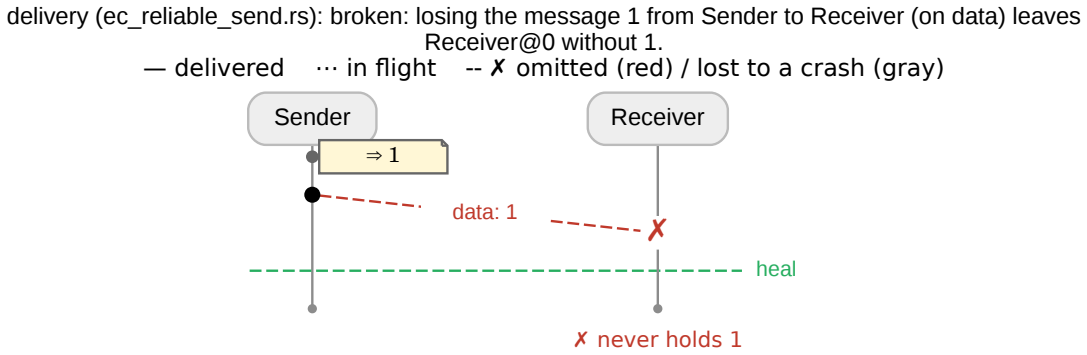


Figure 1: Hydrofrac's counterexample for plain `send` over a lossy link. Each horizontal line is a machine and each arrow is a message, and a red arrow ending in ✗ is a lost message. The sender's only copy of value 1 is lost on the `data` channel before the network heals, and the receiver never holds 1.

```
vals.implies(          // vals: the values handed to the sender, the "if" side of the rule
    "delivery",        // the name that appears in reports
    &delivered,        // the receiver's delivered values, the "then" side of the rule
    q!(|v| v),         // the key of an input value: the value itself
    q!(|v| v),         // the key of a delivered value: the value itself
    ASide::Correct,    // count only the inputs of senders that never crash
    BSide::AllCorrect, // and require every receiver that never crashes to deliver each of them
);
```

Resending until acknowledged

The fix is `reliable_send`, a drop-in replacement for `send`. The sender puts each new value on the wire at once and adds it to an *outstanding set*. The receiver answers every arrival, including duplicates, with an *acknowledgement* (an *ack*), a short message saying that it has the value, and each ack removes its value from the outstanding set. A timer fires once per second, and each time it fires the sender resends every value that is still outstanding. The receiver delivers each value once and discards duplicates. The retry loop is one `sliced!` block in `hydro_std/src/reliable_send.rs`, shown here as it ran, with only the `nondet!` explanations elided. Its body runs once per *tick*, a step in which each `use::batch` takes whatever has arrived on its input since the previous tick:

```
let wire = sliced! {
    let new = use::batch(source, nondet!(/** ... */));          // new values handed to the sender
    let acked = use::batch(acks, nondet!(/** ... */));          // acks (line 79)
    let ticks = use::batch(resend_ticks, nondet!(/** ... */));  // timer firings
    let mut outstanding = use::state_null::<Stream<T, Tick<_>, Bounded, NoOrder>>(); // kept across ticks

    let still = outstanding.filter_not_in(acked);                   // drop acknowledged values
    let resent = still.clone().filter_if(ticks.first().is_some());  // on a timer tick, resend all of them
    outstanding = still.chain(new.clone()).unique();                // carry the rest and the new values forward

    new.chain(resent)                                               // first copies and resends go on the wire
};
```

The outstanding set is the only state that survives from one tick to the next. Every tick on which the timer fires resends the *whole* outstanding set, which is where the resend storm of §2 comes from. The three `use::batch` lines are the inputs that Cress delays in §2, and line 79, the acks batch, is the one named in its report.

Hydrofrac asks the same question of `reliable_send` and gets a different answer. When it cuts the first copy of a value, the rerun still delivers the value, because a later tick resent it. The outcome had a redundant support, which joins the lineage for the next round of the search. Faults are allowed only before the network heals, and afterwards the timer keeps firing, so every fault set leaves some copy that gets through, and when no fault set within its bounds can cut every support, hydrofrac issues a certificate. It certified delivery after 112 fault sets for one value, and after 576 fault sets in 74 seconds for three values whose first copies and acks may be lost in any interleaving. Plain `send` is the control for this experiment: the same harness breaks it with a single loss, so the certificate for `reliable_send` is not an artifact of a search that cannot find losses.

2. Two more questions about the same protocol

Cress: does delay turn into extra work? Cress also runs the program in the simulator, but it never loses a message. Instead it holds back one input batch for longer and longer and compares the number of messages sent with that of a run in which nothing is held. If the extra work stops at a ceiling, delay is harmless, and if it keeps growing with the hold, the program turns delay into load. For `reliable_send`, Cress held the batch of incoming acks for 1, 2, 4 and so on up to 999 rounds of the simulation, and it reported this for the unbudgeted protocol (V0):

```
! HAZARDOUS  reliable_send_pair (two_per_round)  growth: grows  multiplier: 930.01x  location: hydro_std/src/
reliable_send.rs:79, batch of Numbered<u64>
    outputs: conserved: delivered
```

The location is the line of `reliable_send` where the sender takes in acks, whose batch Cress held. The extra work grows with the hold and has no ceiling: at a hold of 999 rounds the sender sends 232,252 more messages than in the unheld run, about 248 extra resends per round of hold, while holding the timer or the new values added nothing. “Conserved: delivered” means that the receiver delivered the same values in every run, so only wasted work grew; Cress labels any extra work HAZARDOUS, and the severity is in the growth and the multiplier.

Sedge: does the deployed program recover from a burst? Sedge deploys the program in Docker containers and imposes a capacity from outside with a CPU quota, so the program contains no capacity code. It first calibrates μ, the receiver's capacity, which is the highest steady rate of new values per second that the receiver keeps up with. It then offers a steady load λ = μ/f for several factors f, so a larger f means a lighter load. The load is open loop (values arrive on a schedule whether or not earlier ones were delivered), with a 10-second baseline, a 5-second burst at twice μ and a 45-second tail, and Sedge compares the tail with the baseline to report RECOVERED, DRAINING or COLLAPSED. Its deployments run over plain TCP and lose nothing. The configuration is an attribute on the function under test:

```
#[cfg_attr(feature = "sedge", sedge::overload_check(
    name = two_per_round,
    T = super::rpc_retry::Payload<4096>,   // 4 KiB values
    load = values,                         // the input that receives the open-loop load
    mu = 500,
    cpus(Sender = none, Receiver = 0.01),  // the receiver gets 1% of a core
    cpu_period_us = 100_000,
    timers(resend_ticks = std::time::Duration::from_secs(1)),
))]
```

With one percent of a core, the receiver's capacity is μ = 500 values per second. We ran two trials at each of f = 2, 4, 8, 16, 32 and 64, so the steady load ranged from 250 down to 8 values per second, and eleven of the twelve trials collapsed. Figure 2 shows a typical one. After the burst the sender writes about 800 messages per second at every load, while the receiver, throttled in 85–100% of its CPU periods, delivers only 2–11% of that traffic as new values. The rest is resends.

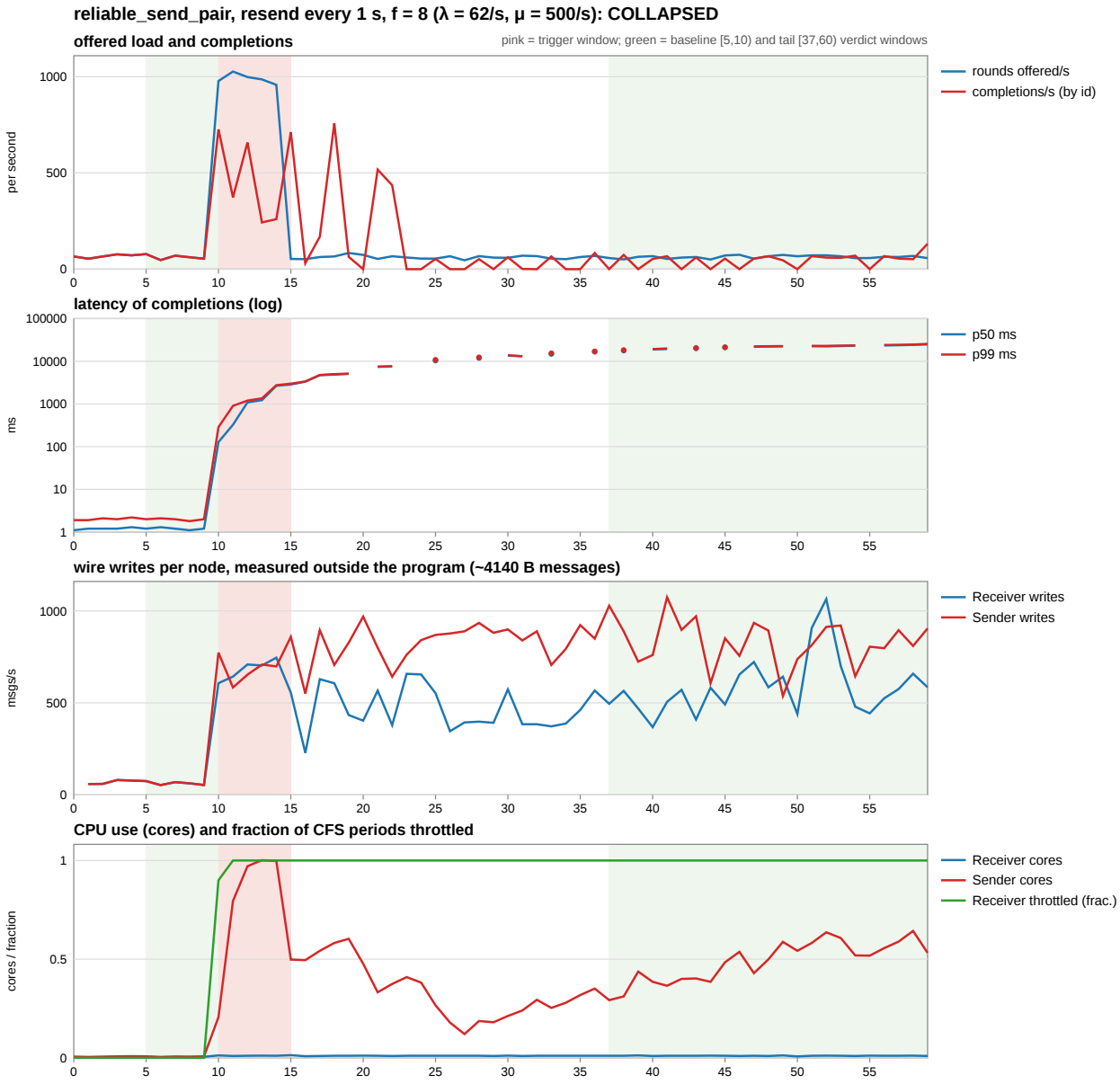


Figure 2: Sedge's trace of `reliable_send` (V0) at f = 8, which offers λ = 62 values per second, with a one-second resend timer. The pink band is the burst, and the green bands are the baseline and tail windows that the verdict compares. After the burst, completions fall to almost nothing, latency climbs past 10 s, the sender keeps writing about 800 messages per second, and the receiver stays throttled.

Figure 3 is the control for Figure 2. It has the same program, quota, timer and steady load, but the "burst" is set to the baseline rate, so the load stays flat. In this single trial Sedge reports RECOVERED: every value offered in the baseline and tail windows completed with a median latency of 1.2 ms, the receiver was throttled in 3% of its CPU periods, and the sender wrote about the offered load throughout. The steady load alone leaves the protocol healthy, so the collapse in Figure 2 comes from the burst and the resend storm that it starts.

Why it collapses. The burst, and not the steady load, sets the size of the storm. During the burst, values arrive at twice the rate that the receiver can acknowledge, so the outstanding set grows to about 2,000 values. Every tick then resends all of them, so the sender asks for at least 2,000 messages per second from a receiver that can absorb about 800. While the receiver works through resends, acks stay late, so the next tick resends everything again. Stretching the resend period to 20 seconds cuts this demand by the same factor, and in one trial each at f = 2 and f = 8 the protocol then recovered within about three seconds of the burst.



Figure 3: The control for Figure 2: `reliable_send` at f = 8 with a one-second resend timer and a flat load of 62 values per second. The pink band marks where the burst falls in the other runs, but here the load does not rise. Completions track the offered load, median latency stays near 1 ms, and the receiver is barely throttled. The vertical axes are scaled to this run, so they differ from those of Figure 2.

3. A budget for resends

A *token bucket* limits a rate: it holds up to B tokens, every resend spends one, a resend that finds it empty waits, and some event refills it. First copies stay free, our buckets start full with B = 10 tokens, and the variants differ only in where tokens come from. V2 earns one token per four acks that clear an outstanding value, and V3, the *hybrid*, also earns one token per timer tick. The bucket adds `tokens` (and the ack credit that earns them) to the state of the `sliced!` block above, and against V0's block it replaces the resend line and the update of the outstanding set. Resends go round robin, so no value waits forever while tokens keep coming:

```
-    let resent = still.clone().filter_if(ticks.first().is_some());
-    outstanding = still.chain(new.clone()).unique();
+    let fired = ticks.count();
+    let available = tokens.zip(tick_grant).zip(ack_grant)   // saved tokens plus this tick's grants, at most B
+        .map(q!(move |((saved, by_tick), by_ack)| (saved + by_tick + by_ack).min(capacity)));
+    let placed = still.enumerate().cross_singleton(available.clone()).cross_singleton(fired)
+        .map(q!(|((position, value), budget), fired)| {
+            (fired > 0 && (position as u64) < budget, value)   // the token check
+        }));
+    let resent = placed.clone().filter_map(q!(|(resend, value)| resend.then_some(value)));
+    let waiting = placed.filter_map(q!(|(resend, value)| (!resend).then_some(value)));
+    let spent = resent.clone().count().map(q!(|n| n as u64));
+    tokens = available.zip(spent).map(q!(|(available, spent)| available - spent));
+    outstanding = waiting.chain(resent.clone()).chain(new.clone()).unique();  // resent values go to the back
```

Table 1: The verdicts of the three tools, with the unbudgeted `reliable_send` labelled V0. The budgeted variants ran in Sedge at f between 1.1 and 16, with μ pinned at 500 so that every variant saw the same load.

variant	hydrofrac: delivery under loss and crashes	Cress: extra work under a held ack batch	Sedge: recovery from the burst
V0, no bucket	Holds.	Grows without a ceiling.	11 of 12 trials collapsed.
V2, refilled by acks	**Broken: a value can be lost forever.**	Stops at a ceiling of 10.	12 of 12 trials recovered.
V3, refilled by acks and the timer	Holds.	Grows at one message per round.	12 of 12 trials recovered.

Table 1 collects the verdicts. A third bucket (V1), which the timer alone refills, also passes all three tools in this setting and differs from the hybrid only on a lossy link, as described below.

The bucket that starves. The ack-refilled bucket V2 looks like the best design to Cress, whose ack hold stops its extra work at ten messages (the bucket size, since no acks arrive to refill it), and to Sedge, where it recovers from every burst. Hydrofrac finds that it can lose a value forever. Figure 4 shows the counterexample for a test bucket of one token, found after three fault sets and 15 ms. The first copy of value 1 is lost, the first tick spends the only token on a resend, and that copy is lost too. No ack can now arrive to earn a token, so the value stays outstanding forever, after two lost messages and no crash. For every bucket from one to four tokens, hydrofrac found the same starvation with exactly B + 1 lost messages, the first copy and the B resends that the bucket pays for. Cress and Sedge cannot see this, because neither of them loses messages.

The bucket that works. The hybrid bucket V3 passes all three tools. Hydrofrac certifies it, and even a hybrid bucket of one token survives the losses that starve V2, because the timer earns the token for the next resend. Under Cress its extra work grows at only one message per round of hold, which is the timer's refill rate. It recovers in all twelve Sedge trials, and Figure 5 shows it on the same arrivals as the collapsed run of Figure 2: it settles within three seconds of the burst, and in the tail the sender writes only the offered load.

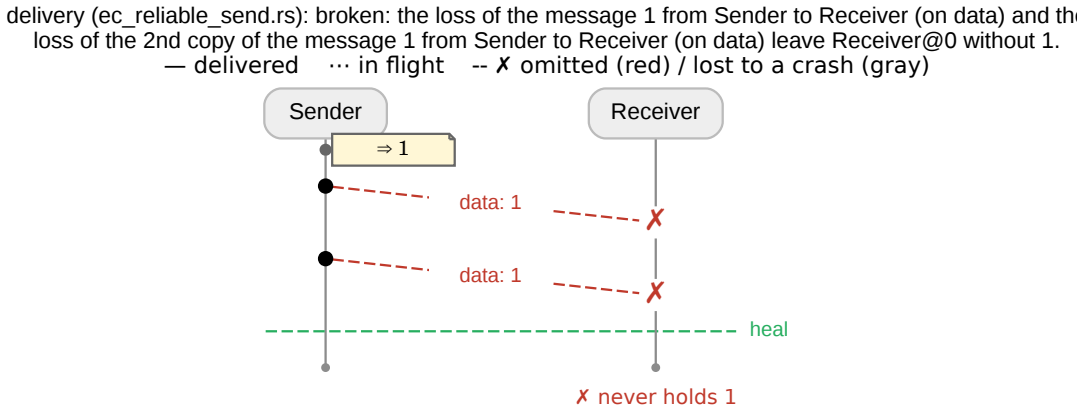


Figure 4: Hydrofrac's counterexample for an ack-refilled bucket of one token (V2). The first copy of value 1 and the one resend that the token pays for are both lost on the `data` channel. No ack ever returns to earn another token, and after the network heals the receiver never holds value 1.

A twist on a lossy link. When Sedge's transport drops any send that finds its 64-message queue full, the ranking changes. V0 recovered best, delivering all of its lost values within 28 seconds, because the full queues shed its resend storm inside the sender. The hybrid bucket recovered lost values at about nine per second, and the bucket refilled by the timer alone, at about one per second, never caught up in a four-minute run. The right budget therefore depends on the transport.

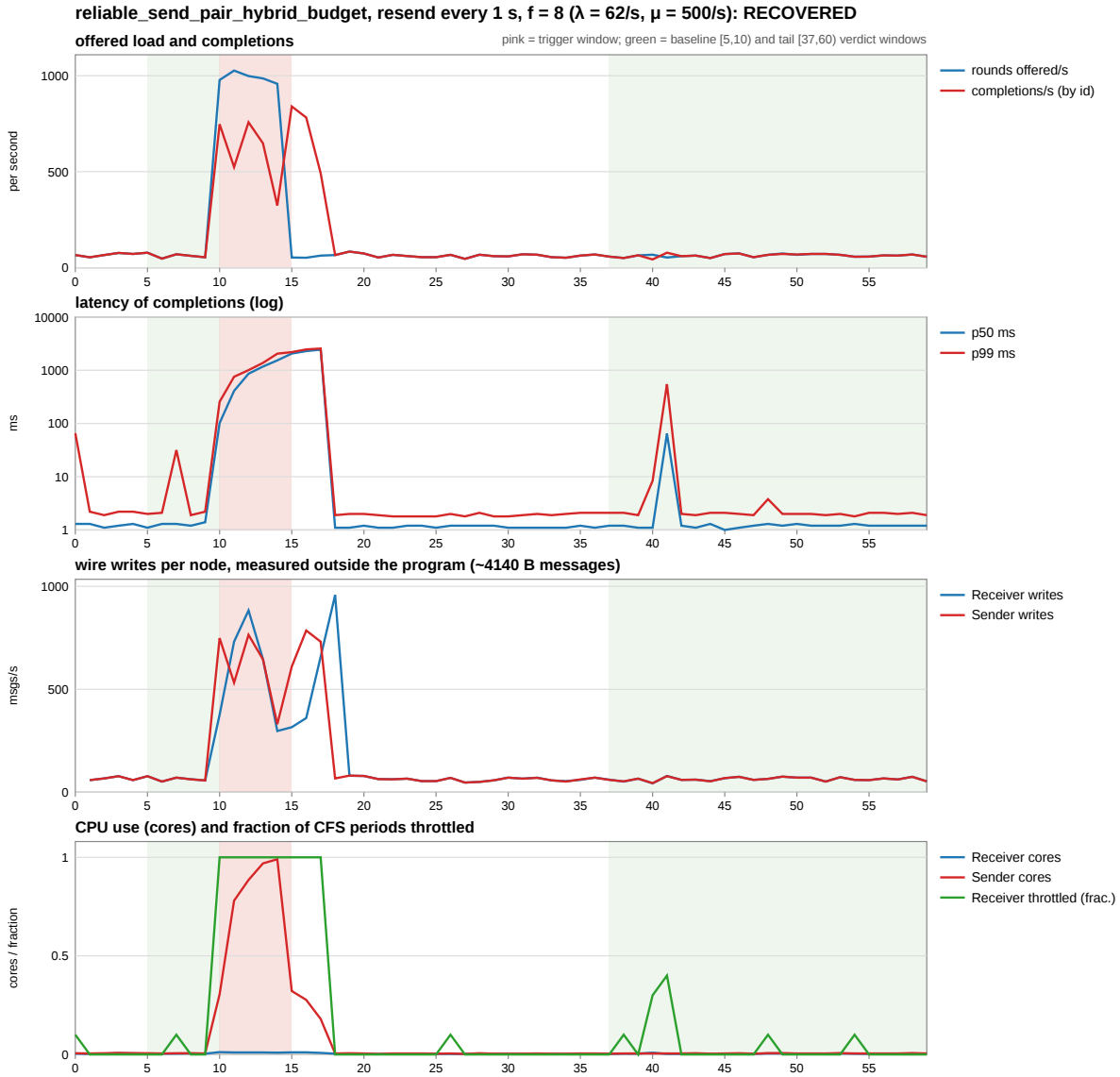


Figure 5: Sedge's trace of the hybrid bucket (V3) at f = 8 with a one-second resend timer, on the same seeded arrivals and the same receiver quota as Figure 2. The latency spike during the burst clears within about three seconds, and in the tail the sender writes only the offered load.

4. What the tools found, and the limits

Each tool caught something that the other two missed, and only the hybrid bucket did well on every test. On identical seeded arrivals, V0 did worse than each budgeted variant in all eight paired trials ($p = 0.0078$, sign test). The results have four limits:

- The Sedge results come from one shared host, two trials per setting and two fixed seeds, and V0 ran in an earlier session.
- Hydrofrac's certificates hold within bounded searches, and for buckets they depend on the harness's recovery time.
- Starvation of the ten-token bucket is extrapolated from searches with one to four tokens.
- Only hydrofrac and the one lossy-transport experiment lose messages.

The full report, `docs/design/reliable-send-three-tools.md`, names every test, run and sweep.